

Classes, Objects, and Methods

Introduction to Object-oriented Programming (OOP)

Object-oriented Programming (OOP) is one of the three (3) programming paradigms, including Procedural and Functional Programming. A **programming paradigm** refers to the different approaches to structuring and organizing code. For context, the term “*paradigm*” is synonymous with “*pattern*”.

In comparison, here is how each programming paradigm works:

Procedural Programming

It is a style of programming in which operations are executed one after another in a sequence. It defines and uses **variables** that refer to the named computer memory locations that can hold data.

For example, data might be read from an input device and stored in a location the programmer has named `percentageOfPay`. This variable value can be used as the basis for a decision, used in an arithmetic statement, or sent to an output device. The data stored in a variable can change during the execution of a program.

Its basic construct is blocks of code named “*procedures*”. **Procedures** are grouped logical units based on individual operations used in a computer program. Procedures are also called *functions*, *modules*, *subroutines*, and *methods*, and *Java programmers most frequently use them*.

As a procedural program executes its statements, it can sometimes pause to **call a procedure, temporarily suspending the current logic so that the procedure’s commands can be executed**.

BASIC, C, C++, and Pascal support this programming paradigm.

Functional Programming

It is a style of programming that builds computer programs by focusing on declarations and expressions rather than the execution of statements. It aims to write code that is clearer to understand and more bug-resistant by avoiding flow-control statements such as `for`, `while`, `break`, `continue`, and `goto`, which makes the code harder to follow.

Functions are the basic units of this programming paradigm, which are also treated like first-class citizens wherein they can be assigned to a variable, passed as an argument (a value or set of values that a function uses to perform its tasks), or returned from a function just like a regular variable.

Erlang, Scala, Haskell, and Elm support this programming paradigm.

Object-oriented Programming (OOP)

It is an extension of procedural programming with a slightly different approach to writing computer programs. **Objects** representing a model of an object in the real world are the main building blocks of OOP.

OOP was developed to eliminate the limitations of other programming paradigms and to make programming more flexible, not too complex, and user-friendly.

Writing OOP involves the following:

- Creating classes that are the blueprint for objects
- Creating objects that are specific instances of those classes
- Creating applications that manipulate or use those objects

Java, Python, VB.NET, and C# support this programming paradigm.

The naming conventions used in Procedural Programming and OOP are likened below:

Procedural Programming	Object-oriented Programming
Variable	Objects
User-defined Data Types	Classes
Structured Members	Instance Variables
Functions	Methods
Function Call	Message Passing

Here are some of the differences between Procedural Programming and OOP.

Procedural Programming	Object-oriented Programming
Emphasizes the procedures rather than the data	Emphasizes the data rather than procedures
Data is not secured	Data is secured
Uses a top-down approach	Uses a bottom-up approach
It does not model the real-world entities	It models the real-world entities
Decomposes programs into functions or procedures	Decomposes programs into objects

Advantages of OOP

- Modularized programs by using classes and objects.
- Reduced code duplication and code reusability as linking code and object allows related objects to share common code.
- Provides data security as the data is encapsulated along with functions, making the data from non-member functions inaccessible and unmodifiable.
- Reduced complexity of the program development through the use of inheritance.
- Reduced time as creation and implementation of OOP code is easy.

OOP Concepts

Class

It is a group or collection of objects with common properties. It is the basic unit of programming in OOP. It is also a user-defined data type, meaning it is created by the user to store complex data.

In layman's terms, it is the same way that a blueprint exists before any houses are built or a recipe exists before any bread is baked from it. A class definition exists before any objects are created from it.

A **class definition** describes what attributes its objects will have and what those objects can do. An **attribute** details the characteristics that define an object, which serves as **properties** of the object.

Object

It is a specific and concrete *instance* of a class. It is an identifiable entity with some characteristics and behavior, which can be a person, place, or table of data. It is any real-world entity that has its existence and has both features called **data members** and operations called **function members**.

Using a smartphone to further visualize the difference between data members and function members, the features of a smartphone, such as its color, weight, and price, are considered the data members, while its operation to make a video call, take photographs, or record audio are its function members.

Method

Other than defining properties, classes define the methods their objects can use. A **method** is a self-contained block of program code that carries out actions similar to a procedure in Procedural Programming.

Here are some examples that interconnect the three (3) concepts.

Class	Object	Method
Automobile	Make, Model, Year, Color	Forward, Backward, Gas Status
Dog	Breed, Name, Age, Vaccine	Walking, Eating, Name Recognition

Understanding the object's class helps in understanding the characteristics of the object in the sense that if a friend purchased an automobile (class), it is a given that it has a model name (object.)

Additionally, objects can be compared to nouns, while methods are similar to verbs.

Data Abstraction

It is the act of including only essential details of an entity without including the background details.

For example, a smartphone with Bluetooth and camera features. Data abstraction does not care how Bluetooth or the camera works; it is only interested in using the smartphone and its features rather than their internal working. That information is hidden from the user and is called **abstracted data**.

Data Encapsulation

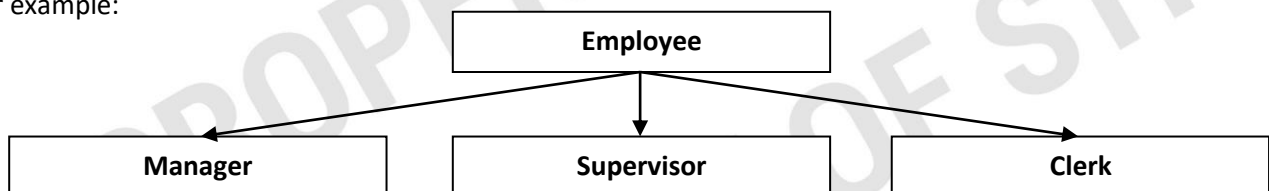
It is the process of wrapping up data and functions into a class. It performs **data hiding**, which insulates data from outside programs. The data would not be accessible directly to the outside world except for the functions defined inside the class definition.

Inheritance

It is the ability to create classes that share the attributes and methods of existing classes but with more specific features.

The class that inherits the properties of the other class is called a **base class**, **parent class**, or **superclass**. The class that inherits the properties from the other class is called a **derived class**, **child class**, or **subclass**.

For example:



Employees are the base class, while the others below them are the derived class. The employee base class inherits properties from the derived classes, with each derived class having its own properties.

Polymorphism

Poly means *many*, while *morph* means *forms*. It describes the feature of languages that allows the same word or symbol to be interpreted correctly in different situations based on the context. Examples of this include operator overloading and function overloading.

Operator Loading is the process of making an operator perform tasks in different instances. For example, using an addition (+) operator. As an arithmetic operator, it performs addition, but if used in a String such as "Just"+"ice", it combines the two (2) strings and acts as a String concatenation operator resulting in "Justice".

Function Overloading is the process of two (2) or more functions with the same name but different return types or numbers of arguments. For example, "Paint() Items" as a single interface but has many methods, such as paint() furniture, paint() vehicle, and paint()building.

Methods

A **method** is a program module containing a series of statements that carry out a task, which can be called an unlimited number of times. A familiar example is the `main()` method, which executes automatically when a program is run. A program's `main()` method can execute additional methods, which can also execute others.

To execute a method, a programmer calls it or invokes it. In other words, a **calling method**, or *client method*, invokes a **called method**.

For example, a class is created to display the organization's name in a single line of output.

```
public class CompanyInfo {  
    public static void main(String[] args) {  
        System.out.println("Smart Solutions Electronics");  
    }  
}
```

For additional context, a **method header**, or *method declaration*, provides information about how other methods can interact with it. In this example, `public static void main()` is the method header.

- **public** – an **access modifier** that allows any other class to use it and not just in the class in which the method resides. Other access modifiers include `private`, `protected`, and `package` if left unspecified.
- **static** – used when any method can be used without instantiating an object or not requiring an object when they are called.
- **void** – a return type used when a method returns no data. A **return type** describes the data type the method sends back to its calling method.
- **Method Name** – can be any legal identifier for classes and variables. It must be one word with no spaces and cannot be a Java keyword. Methods perform an action, so their names usually contain a verb, such as `display` or `compute`.
- **Parentheses** – contains data to be sent to the method. When a `main()` method is written in a class, the parentheses in its header surround `String[] args`.

A **method body** contains the statements that carry out the work of the method. It is found between a pair of curly braces. The body of a method is called its **implementation**.

Assume three (3) lines of output are added to this application to display the business hours of the company; this is how it will look like:

```
public class CompanyInfo {  
    public static void main(String[] args) {  
        System.out.println("Smart Solutions Electronics");  
        System.out.println("Monday - Friday 7 AM to 5 PM");  
        System.out.println("Saturday 7 AM to 12 NN");  
        System.out.println("Sunday Closed");  
    }  
}
```

But instead of adding three (3) `println()` statements, a method can be called to execute the statements. Using the `displayHours()` method, the program will now look like this:

```
public class CompanyInfo {
    public static void main(String[] args) {
        System.out.println("Smart Solutions Electronics");
        displayHours();
    }
}
```

Additionally, a method must be actually written in the program. It can be placed within a class, such as the CompanyInfo class, but it must be outside of any other methods. A method cannot be placed within another method, such as the details of the displayHours() method cannot be placed inside the main() method of the CompanyInfo class. Methods can also never overlap.

```
public class CompanyInfo {
    // A method can be placed here, before the main() method
    public static void main(String[] args) {
        System.out.println("Smart Solutions Electronics");
        displayHours();
    }
    // A method can also be placed here, after the main() method
}
```

The order in which the methods appear in a class does not matter on the order in which they are called or executed. The main() method is always executed first in any Java application. On the other hand, the order in which the methods are called is what makes a difference in how an application executes.

This is the complete CompanyInfo class with the defined displayHours() method.

```
public class CompanyInfo {
    public static void main(String[] args) {
        System.out.println("Smart Solutions Electronics");
        displayHours();
    }
    public static void displayHours() {
        System.out.println("Monday - Friday 7 AM to 5 PM");
        System.out.println("Saturday 7 AM to 12 NN");
        System.out.println("Sunday Closed");
    }
}
```

A complete name that includes the class is a **fully qualified identifier**. It includes the class name, a dot, and the method name. The full name of the displayHours() method is CompanyInfo.displayHours(). This is necessary if the same method is used in another class, as the compiler does not recognize the method unless its full name is used.

Advantages of creating a separate method:

- Using a method call to execute different println() statements makes the main() method short and easy to follow.
- Using a well-named method makes it easy to see the overall intent of the separate println() statements.
- As methods are easily reusable, they can be used in any application that needs that specific action in the program.

Classes and Objects

Java classes can be classified into the following types:

- Classes from which objects are not instantiated, such as the programs with the main() methods
- Classes from which objects are created

With OOP, a class can be created to run as an application to instantiate objects from them and do both.

When creating a class, a name must be assigned to it, and the data and methods that will be part of the class must be determined.

A **class header** has three (3) parts: an optional access specifier, the keyword `class`, and any legal identifier in naming the class, starting with an uppercase letter. For example, `public class Employee`

After the class header, the **class body** is written between a set of curly braces and contains the data and methods for the class. The data components of a class are called **data fields** that are declared variables within a class but outside any method.

```
public class Employee {  
    private int empNum;  
}
```

`empNum` is the data field in this example. It is not preceded by the keyword `static`. If it had been added, a single `empNum` value would have been shared by all employee objects, which would eventually be instantiated. Hence, in this code, each `Employee` can have a unique `empNum`. A data field is **static** if it occurs once per class and **non-static** if it happens once per object.

The `private` access specifier is used here, providing the highest security level. Assigning private access to a field means that no other classes can access the field's values except the methods of the same class. The principle used in creating private access is referred to as **information hiding**, an important component of object-oriented programs.

Classes that instantiate objects contain both fields and methods. For methods, the method needed for the `Employee` class with an `empNum` is the one that returns any `Employee`'s `empNum` for use by another class. This can be named `getEmpNum()`. Methods that retrieve values are called **accessor methods** or **getters** that conventionally start with the prefix `get`.

Since it will have public access, return an integer (the employee number), and possess the identifier `getEmpNum`, it will be declared as `public int getEmpNum()`. It is not `static` if the intention is for each `Employee` object to have its own `empNum` value.

Additionally, a method is needed to set the `empNum` field. This can be named `setEmpNum()` with a declaration of `public void setEmpNum(int emp)` as it will have public access, return nothing, have the `setEmpNum()` identifier, and require a parameter to represent the employee's ID number, a data type `int`. Methods that set or change field values are called **mutator methods** or **setters** that conventionally start with the prefix `set`.

```
public void setEmpNum(int emp) {  
    empNum = emp;  
}  
  
public int getEmpNum() {  
    return empNum;  
}
```

In declaring objects, a two-step process is involved in creating an object that is an instance of a class. First, a type and an identifier are supplied, just like declaring any variable; second, computer memory is allocated for that object.

For example, an integer as `int someValue;` can be declared this way, and an `Employee` can be declared as `Employee someEmployee;` wherein `someEmployee` can be any legal identifier, but objects start with a lowercase letter.

In declaring `int someValue`, the compiler is notified that an integer named `someValue` will exist, and computer memory will be reserved for it at the same time. In declaring `Employee someValue`, the compiler is notified that the identifier `someEmployee` will be used, but computer memory will not be reserved for it.

A new operator should be used to allocate the memory for an object such as this.

Two (2) statements that complete the process by setting aside enough memory to hold an `Employee` are as follows:

```
Employee someEmployee;
```

```
someEmployee = new Employee;
```

This can be shortened by declaring and reserving memory for `someEmployee` in one statement:

```
Employee someEmployee = new Employee();
```

Wherein:

- `Employee` is the object's type and class, while `someEmployee` is the object's name.
- `Employee` now also becomes a *reference type* as opposed to built-in types like `int`, which are *primitive types*.
- `someEmployee` also becomes a reference to an object – the name for a memory address where the object is held.
- The equal sign (=) is the assignment operator, assigning a value to `someEmployee` in the declaration.
- The new operator is allocating an unused portion of computer memory for `someEmployee`.
- `Employee()` after the new operator is the name of the method that constructs an `Employee` object. It is a constructor.

A **constructor** is a special type of method that creates and initializes objects. A constructor can be user-specified, but Java also writes one whenever a user does not write one. The name of the constructor is the same as the name of the class whose objects it constructs.

After an object has been instantiated, the methods can be accessed using the object's identifier, a dot, and a method call.

```
public class DeclareTwoEmployees {
    public static void main(String[] args) {
        Employee clerk = new Employee();
        Employee driver = new Employee();
        clerk.setEmpNum(345);
        driver.setEmpNum(567);
        System.out.println("The clerk's number is " +
            clerk.getEmpNum() + " and the driver's number is " +
            driver.getEmpNum());
    }
}
```

The program above shows the `DeclareTwoEmployees` application that instantiates two (2) `Employee` objects, `clerk` and `driver`, with each using `setEmpNum()` and `getEmpNum()` methods one time.

References:

Farerel, J. (2023). *Java programming, 10th edition*. Cengage.

Ladwa, H. (2021). *Object-oriented programming with Java*.

Mitchell, B.. (2022). *What is functional programming and why use it?* [Web Article]. Retrieved on October 24, 2024, from <https://www.codingdojo.com/blog/what-is-functional-programming>

Scarler Topics (2022). *OOP vs functional vs procedural*. [Web Article]. Retrieved on October 30, 2024, from <https://www.scaler.com/topics/java/oop-vs-functional-vs-procedural/>